Oracle Database 19c RAC 과정

RAC 개요와 아키텍처

하나의 데이터베이스를 여러 인스턴스가 함께 연다

제1장

학습 목표

- 수평 확장과 RAC가 막는 장애·막지 못하는 장애를 구분한다
- 노드·인터커넥트·공유 스토리지와 세 소프트웨어 계층을 설명한다
- 공유 파일과 인스턴스별 구성 요소를 나열한다
- GV$ 뷰로 클러스터 전체를 조회한다
- Cache Fusion과 gc 대기 이벤트를 설명한다
- 공용 IP·VIP·SCAN·사설망의 용도를 구분한다

Agenda

- 1.2 확장성과 가용성
- 1.3 클러스터 구성 요소 / 1.4 소프트웨어 스택
- 1.5 하나의 데이터베이스, 여러 인스턴스
- 1.6 Cache Fusion / 1.7 네트워크 구성
- 1.8 실습 환경 / 1.9 라이선스
- 1.10 인스턴스 하나가 빠지면

1.2

확장성과 가용성

- 수직 확장과 수평 확장
- 계획되지 않은 정지와 계획된 정지
- RAC가 막지 못하는 것

수직 확장과 수평 확장

RAC는 서버를 더 붙여 확장한다

- 수직 확장: 더 큰 서버로 교체, 상한과 비용 문제
- 수평 확장: 같은 사양 노드를 클러스터에 추가
- 데이터를 쪼개지 않음, 어느 노드에서든 전체가 보임
- 같은 블록을 심하게 갱신하면 선형으로 늘지 않음

계획되지 않은 정지와 계획된 정지

노드 하나가 멈춰도 데이터베이스는 열려 있다

- 하드웨어 고장·OS 패닉에도 남은 노드가 서비스
- 살아 있는 인스턴스가 죽은 쪽 리두로 인스턴스 복구
- 끊긴 세션의 재접속은 TAF·AC의 몫 (13장)
- 패치·점검은 노드별 롤링 작업으로

RAC가 막지 못하는 것

스토리지와 논리 오류는 다른 수단으로

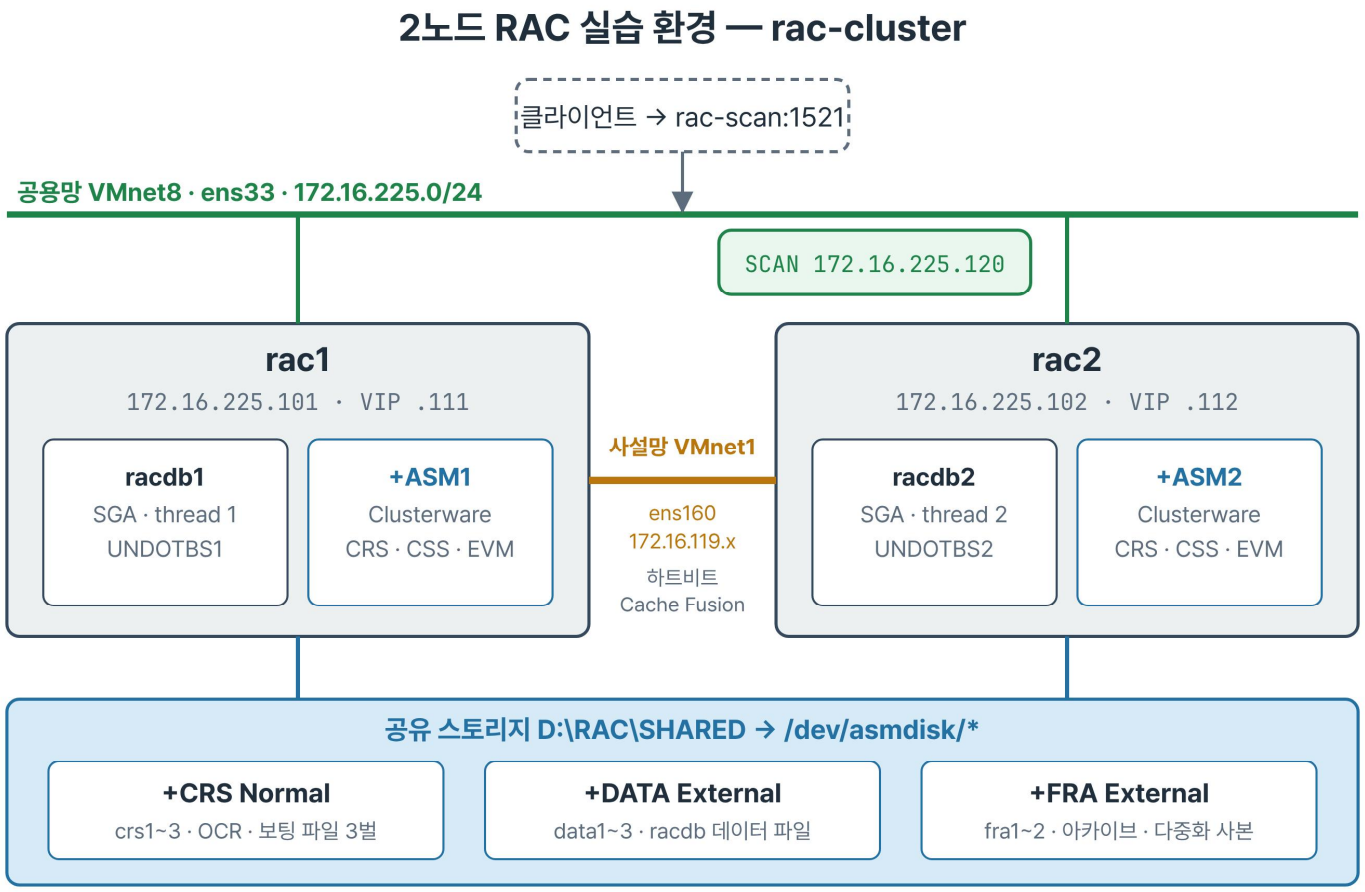
- 모든 노드가 같은 스토리지를 공유
- 스토리지 손상·사이트 정지 → Data Guard
- 잘못된 DELETE·배치 오류 → 백업·플래시백
- 세 가지는 서로를 대신하지 않음

1.3

클러스터 구성 요소

- 노드
- 인터커넥트
- 공유 스토리지

실습 클러스터 rac-cluster



공용망·사설망으로 연결된 두 노드와 공유 디스크그룹 세 개

노드

노드마다 인스턴스가 하나씩

- 노드마다 CPU·메모리·OS·Oracle 홈을 따로 가짐
- 같은 OS·패치·Oracle 버전 유지
- 같은 사양 권장, 느린 노드가 전체를 끌어내림
- racdb는 관리자 관리형: rac1=racdb1, rac2=racdb2

인터커넥트

지연이 튀면 안 되는 네트워크

- 노드끼리만 쓰는 전용망: ens160, 172.16.119.0/24
- 하트비트와 Cache Fusion 트래픽
- 하트비트가 끊기면 노드 축출(eviction)
- 백업 등 다른 트래픽을 태우지 않음

공유 스토리지

모든 노드가 같은 디스크를 같은 이름으로

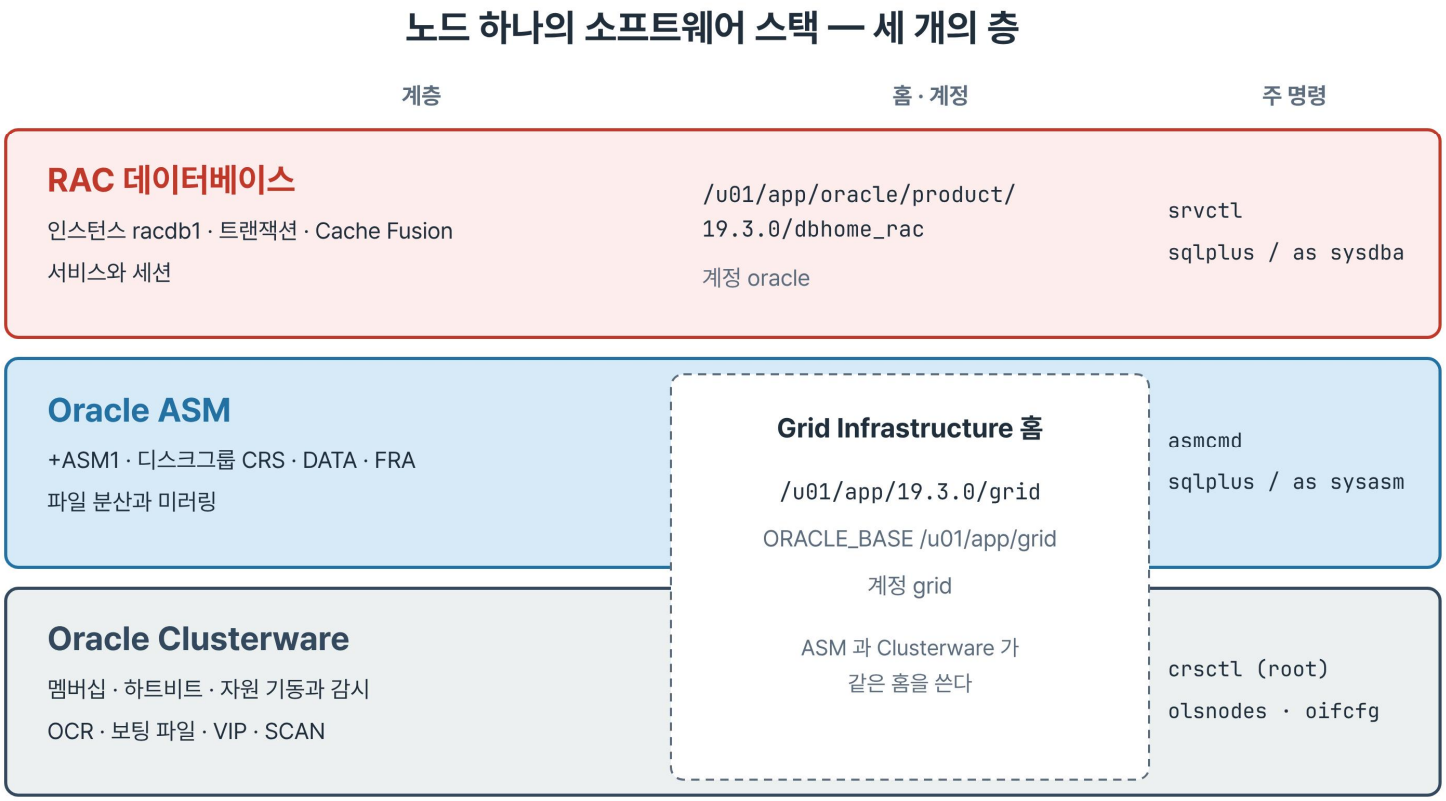
- 데이터 파일·컨트롤 파일·리두·SPFILE·OCR·보팅 파일
- 모든 노드가 동시에 읽고 쓸 수 있어야 함
- /dev/sdX 대신 고정 이름 /dev/asmdisk/*
- +CRS(Normal) · +DATA · +FRA(External)

1.4

소프트웨어 스택

- Clusterware
- Grid Infrastructure
- RAC 데이터베이스

세 개의 층



Clusterware와 ASM은 Grid 홈, 데이터베이스는 DB 홈

Clusterware와 Grid Infrastructure

클러스터를 하나로 묶는 층

- 멤버십·생존 감시·자원 기동과 재기동
- 자원: 리스너·VIP·ASM·DB 인스턴스·서비스
- 스플릿 브레인을 막는 통제자
- GI = Clusterware + ASM, /u01/app/19.3.0/grid

계층별 명령

운영 작업은 srvctl로

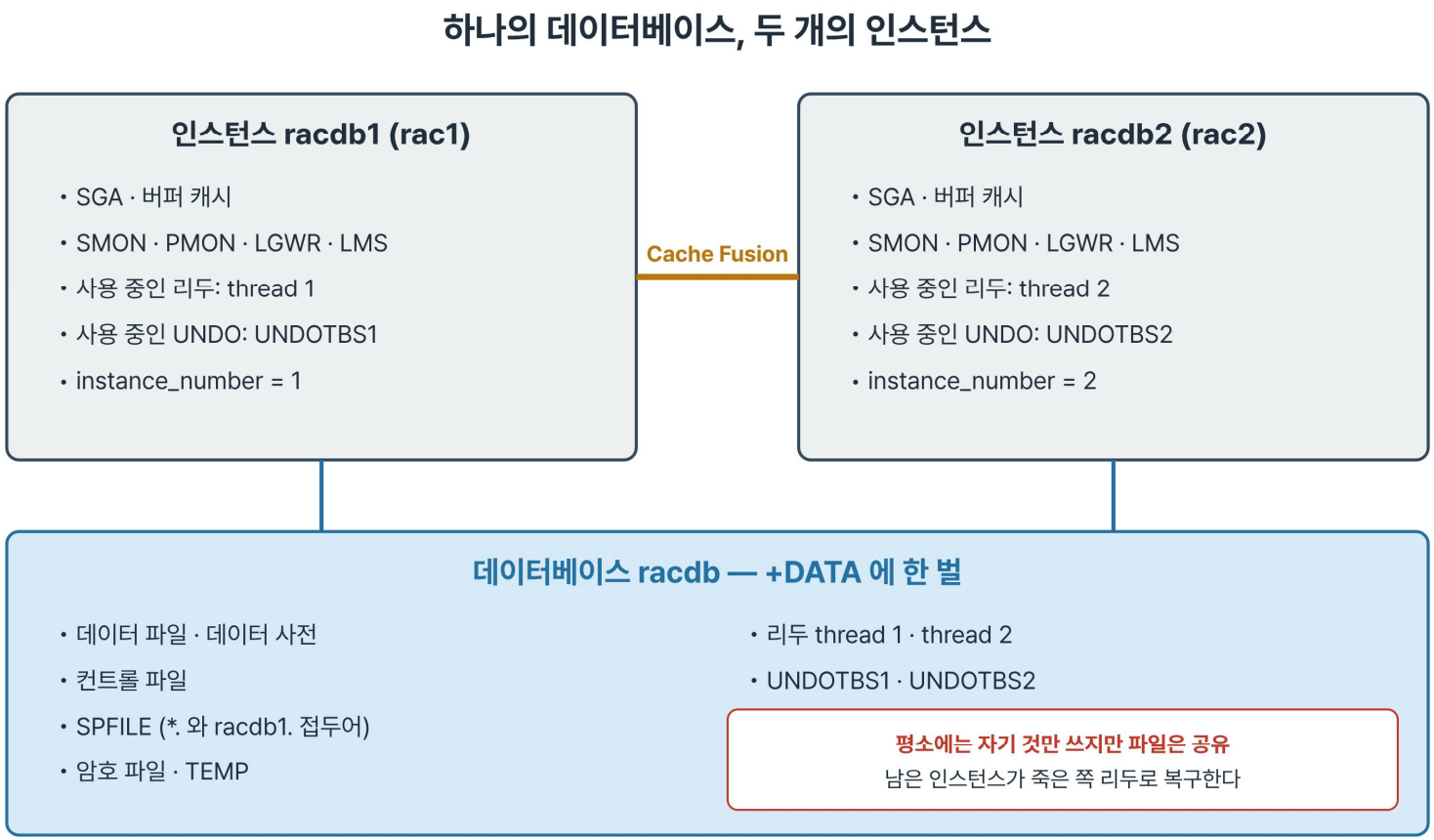
- crsctl: Clusterware 자체, 저수준 자원, 일부 root
- srvctl: DB·인스턴스·리스너·서비스
- asmcmd, sqlplus / as sysasm: ASM
- crsctl로 DB 자원을 직접 내리면 의존 관계를 건너뜀

1.5

하나의 데이터베이스, 여러 인스턴스

- 공유되는 것과 따로인 것
- GV$ 뷰
- RAC 구성 확인 조회

하나의 데이터베이스, 두 개의 인스턴스



리두 스레드와 UNDO도 파일은 공유 스토리지에 있다

GV$ 뷰

V$는 접속한 인스턴스만 보여 준다

- GV$ 뷰에는 INST_ID 열이 붙음
- 다른 인스턴스에 슬레이브를 띄워 결과를 모음
- 응답 없는 인스턴스가 있으면 조회가 멈출 수 있음
- V$만 보고 이상 없음으로 판단하지 않기

RAC 구성 확인 조회

설치 결과를 데이터베이스 안에서 읽는다

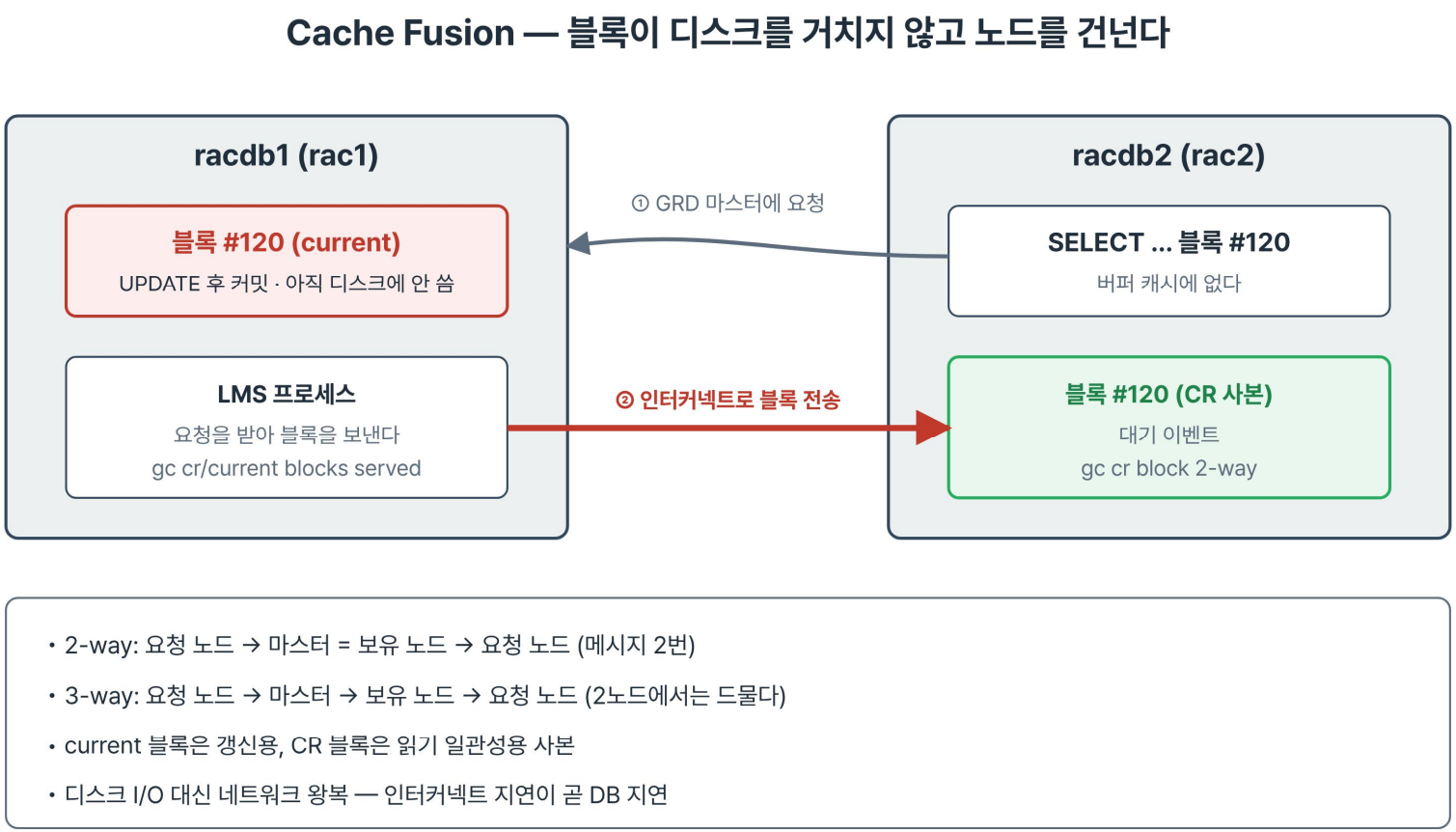
- gv$instance: 두 인스턴스, THREAD# 1과 2
- v$thread: 스레드 2개 OPEN·PUBLIC
- gv$parameter: undo_tablespace, instance_number, cluster_database
- gv$pdbs: RACPDB1이 양쪽에서 READ WRITE

1.6

Cache Fusion

- GRD와 마스터
- 2-way와 3-way
- current 블록과 CR 블록
- gc 대기 이벤트와 통계

Cache Fusion



디스크를 거치지 않고 인터커넥트로 블록을 받는다

GRD와 마스터

블록이 어디 있는지 아는 분산 목록

- GRD: 어느 블록을 누가 어떤 모드로 가졌는지
- 모든 인스턴스가 나눠서 관리
- 마스터 인스턴스에 먼저 묻는다
- GCS는 블록, GES는 그 밖의 잠금, LMS가 전송

2-way와 3-way, current와 CR

무엇을 몇 번 만에 받는가

- 2-way: 마스터가 블록도 가진 경우, 메시지 2번
- 3-way: 마스터와 보유 노드가 다른 경우
- current: 갱신용 최신 블록, 한 인스턴스만 배타
- CR: 읽기 일관성용 사본, 보유 노드가 만들어 보냄

gc 통계와 대기 이벤트

앞뒤 값을 비교해 방향을 읽는다

- gc cr/current blocks received · served
- 누적값이므로 작업 전후 두 번 조회
- 한쪽 received 증가 = 다른 쪽 served 증가
- 대기 증가 원인: 인터커넥트, hot block, LMS CPU

1.7

네트워크 구성

- 공용 IP와 VIP
- SCAN
- 사설 인터커넥트

네 종류의 주소

어느 주소로 접속하느냐가 장애 때 동작을 정한다

- 공용 IP 172.16.225.101/102: 관리 접속
- VIP 172.16.225.111/112: 노드 리스너, 장애 시 이동
- SCAN 172.16.225.120: 클라이언트 접속 이름
- 사설 172.16.119.101/102: 하트비트·Cache Fusion

VIP

TCP 타임아웃을 기다리지 않게 한다

- VIP는 Clusterware 자원(ora.rac1.vip)
- 노드가 죽으면 살아 있는 노드로 이동
- 옮겨 간 VIP에는 리스너가 없어 즉시 거절
- 인스턴스만 내려갔을 때는 옮겨 가지 않음

SCAN

클라이언트는 이름 하나만 안다

- 클러스터 전체를 가리키는 단일 접속 이름
- 운영은 DNS에 IP 3개, 실습은 dnsmasq로 1개
- 인스턴스가 remote_listener로 SCAN 리스너에 등록
- 접속 문자열: rac-scan:1521/서비스명

1.8

실습 환경과 라이선스

- 설치 가이드 v2의 구성 그대로
- 계정과 프롬프트
- RAC 옵션의 경계

실습 환경

설치가 끝난 상태에서 시작한다

- Grid 홈 /u01/app/19.3.0/grid, 계정 grid
- DB 홈 /u01/app/oracle/product/19.3.0/dbhome_rac
- racdb · racpdb1, 관리자 관리형, policy AUTOMATIC
- 실습용 추가 디스크 /dev/asmdisk/test1~6

계정·프롬프트·스냅샷

어느 노드의 어느 계정인지 먼저 본다

- grid: ORACLE_SID +ASM1 / +ASM2
- oracle: ORACLE_SID racdb1 / racdb2
- 공유 디스크는 VM 스냅샷에 들어가지 않음
- ⚠ 실습 전 -BackupShared 사본 확보

라이선스와 옵션

RAC는 EE의 유료 옵션

- RAC: Enterprise Edition 별도 옵션, 노드 전체에 필요
- RAC One Node: 별도 옵션 (16장)
- Clusterware·ASM: DB 라이선스 서버에서 추가 비용 없음
- V$OPTION, DBA_FEATURE_USAGE_STATISTICS로 확인

1.10

인스턴스 하나가 빠지면

- 계속되는 것
- 끊기는 것
- 접속 오류 구분

racdb2가 빠졌을 때

rac2 의 인스턴스가 빠지면

rac1 — racdb1 계속 서비스

- 자기 세션의 트랜잭션은 그대로
- SMON 이 thread 2 리두를 읽어 인스턴스 복구
- 복구 동안 잠긴 블록만 잠시 대기
- 서비스가 racdb1 로 옮겨 온다 (12장)

thread 2

rac2 — racdb2 정지

- 붙어 있던 세션은 끊긴다
- 커밋 전 트랜잭션은 롤백
- 노드가 죽으면 rac2-vip 가 rac1 로 옮겨 가 접속 시도를 즉시 거절
- Clusterware 는 계속 동작 (인스턴스만 정지)

클라이언트에서 보이는 것

- rac-scan 으로 새로 접속하면 racdb1 로 연결된다
- rac2-vip 로 직접 접속하면 ORA-12514 등으로 실패 — VIP 로 접속 문자열을 고정하지 않는다
- 끊긴 세션을 자동으로 이어 주는 것은 TAF · AC 의 몫 (13장)
- 다시 기동하면 racdb2 가 클러스터에 합류하고 서비스는 수동으로 되돌린다

racdb1이 인스턴스 복구를 하고 서비스를 이어 간다

접속 오류 구분

오류 번호가 원인의 위치를 알려 준다

- ORA-12514: 리스너는 있고 서비스가 없음
- ORA-12541: 그 주소에 리스너가 없음
- ORA-12170 / 12543: 노드가 꺼져 연결 불가
- 다시 올려도 옮겨 간 서비스는 자동 복귀하지 않음

요약

- RAC는 한 데이터베이스를 여러 인스턴스가 연다. 사이트 장애·논리 오류는 별도 대비
- 노드·인터커넥트·공유 스토리지, Clusterware·ASM·DB 세 층
- 파일은 한 벌, 리두 스레드와 UNDO는 인스턴스별
- 클러스터 전체는 GV$ 뷰로 조회
- Cache Fusion은 인터커넥트로 블록을 받는다
- 클라이언트는 SCAN, VIP는 타임아웃 방지

퀴즈

- 1. RAC가 막지 못하는 장애 두 가지와 대비 수단은?
- 2. racdb1이 racdb2의 인스턴스 복구를 할 수 있는 이유는?
- 3. V$INSTANCE와 GV$INSTANCE의 결과 행 수 차이는?
- 4. gc cr block 2-way 대기가 늘 때 의심할 원인 세 가지는?
- 5. 인스턴스만 내려간 노드의 VIP로 접속하면 나는 오류는?
- 6. 실습 환경의 SCAN 리스너가 하나뿐인 이유는?

실습 개요

- 01 클러스터 상태 파악 — crsctl status resource -t 읽기
- 02 공유 데이터베이스 확인 — GV$와 V$, 공유 파일
- 03 ★ Cache Fusion 관찰 — gc 통계 전후 대조
- 06 ★ 노드 정지와 복귀 — racdb2 정지 후 되살리기
- 파일: RAC/01장/1장_실습_NN_*.txt